Machine-Checked Linear Temporal Logic for the UEFI Capsule Update Process

A Foundational Proof of Authenticity, Anti-Rollback, and Responsiveness

Vincent Zimmer

IEEE Cyber Awareness and Research Symposium (CARS 2026)

2026

Platform Firmware: Root of Trust Under Active Attack

The Trust Stack Foundation:

- Boot ROM & SPI Flash execute before OS/Hypervisor.
- Establishes measurements for Secure Boot & TPM.
- Subverted firmware persists below disk encryption and survives complete OS reinstallation.

In-the-Wild Threat Landscape:

- *Thunderstrike, LoJax, BlackLotus* (UEFI rootkits).
- *BootHole, PixieFail* (Reference stack bugs).

NIST Standards Mandates (SP 800-193)

1. **Authenticity:** Only cryptographically signed updates are applied.
2. **Anti-Rollback:** Version monotonicity must defeat replay & downgrade attacks.

The Problem

Security arguments currently exist only in **English prose** and **C code** (`FmpDevicePkg`).

The Gap: Informal Prose vs. High-Stakes Code

- **Standard UEFI Capsule Flow:**

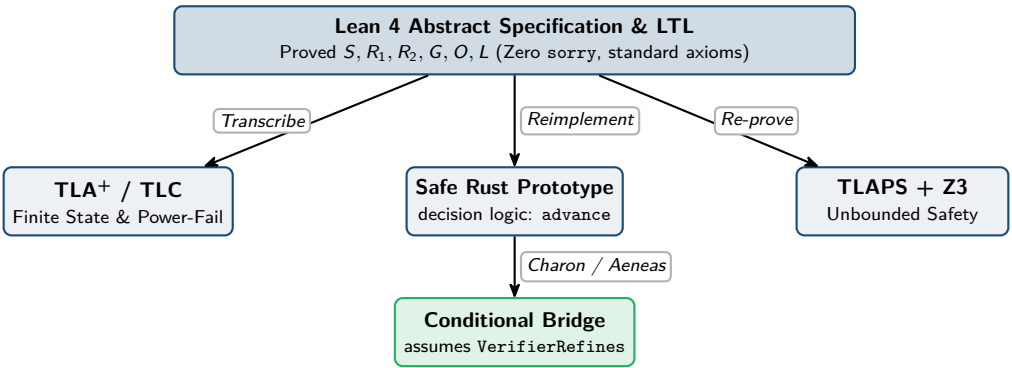
$$\text{OS Stage Capsule} \xrightarrow{\texttt{UpdateCapsule()}} \text{Reset} \xrightarrow{\text{Coalesce}} \text{FMP CheckImage()} \xrightarrow{\text{SetImage()}} \text{Flash Write}$$

- **Why Prose Specifications Fail:**
  - Ambiguities in temporal phrasing (e.g., when is an update "strictly newer"?).
  - Lack of machine-checked invariants across arbitrary sequences of adversarial attempts.
- **Our Opportunity:**
  - Capsule update has a compact, well-defined state machine.
  - Ideal target for interactive theorem proving (following *seL4*, *CompCert*, *Ironclad*).

Goal

Machine-check the capsule-update **decision logic**, audit the proofs (zero `sorry`), and relate a **Safe Rust prototype** to the model under an explicit verifier assumption.

Artifact Verification Chain



System Model & The Phase Automaton

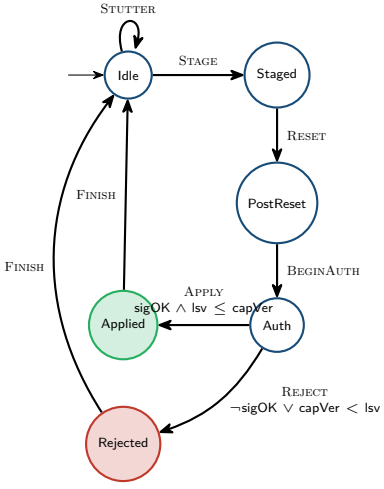
State Tuple $s \in \mathcal{S}$:

$$s = \langle \text{phase},\ \text{fwVer},\ \text{lsv},\ \text{capPresent},\ \text{capVer},\ \text{digest},\ \text{sigOK},\ \text{reset} \rangle$$

- phase $\in \{\text{Idle}, \text{Staged}, \text{PostReset}, \text{Auth}, \text{Applied}, \text{Rejected}\}$
- $\text{fwVer}, \text{lsv}, \text{capVer}, \text{digest} \in \mathbb{N}$
- $\text{capPresent}, \text{sigOK}, \text{reset} \in \mathbb{B}$

Adversary Model:

- Attacker controls capVer and the staged digest.
- BeginAuth sets sigOK := certOK(digest).
- `certOK` abstracts complete capsule authentication.



Shallow LTL Embedding in Lean 4

- **Shallow Temporal Proposition:** A predicate over runs ($\mathbb{N} \to \mathcal{S}$) and positions:

$$\text{TProp} \triangleq (\mathbb{N} \to \mathcal{S}) \to \mathbb{N} \to \text{Prop}$$

Temporal Modalities

$$[\![\text{now } p]\!](\sigma, i) \triangleq p(\sigma\, i)$$
$$[\![\mathbf{X}\ \varphi]\!](\sigma, i) \triangleq [\![\varphi]\!](\sigma, i{+}1)$$
$$[\![\mathbf{G}\ \varphi]\!](\sigma, i) \triangleq \forall j \geq i.\ [\![\varphi]\!](\sigma, j)$$
$$[\![\mathbf{F}\ \varphi]\!](\sigma, i) \triangleq \exists j \geq i.\ [\![\varphi]\!](\sigma, j)$$

Validity Judgment

$$\vDash \varphi \triangleq \forall \sigma.\ \text{IsRun}(\sigma) \to [\![\varphi]\!](\sigma, 0)$$

Key Architectural Advantage

- No external solver or deep AST AST interpreter.
- Dualities (e.g. $\mathbf{F}\,\varphi \equiv \neg\,\mathbf{G}\,\neg\varphi$) are native meta-theorems.
- Tactics like `omega`, `induction`, and `cases` discharge proofs directly.

Core Guarantees (Apply Guard Next)

ID	Class	Formal Statement & Security Meaning
S	Safety	$\models \mathbf{G}\,(\text{now}\,(\text{phase}{=}\text{Applied}) \Rightarrow \text{now}\,\text{sigOK})$ *Authenticity: Unauthenticated images are never applied.*
R1	Safety	$\models \mathbf{G}\,(\text{now}\,\text{lsv} \leq \mathbf{X}\,(\text{now}\,\text{lsv}))$ *Local Anti-Rollback: The effective floor never drops.*
R2	Safety	$\forall\sigma.\ \mathrm{IsRun}(\sigma) \rightarrow \forall i \leq j.\ \text{lsv}(\sigma\, i) \leq \text{lsv}(\sigma\, j)$ *Global Anti-Rollback: The floor is lifetime-monotone.*
F	Safety	$\models \mathbf{G}(\text{lsv} \leq \text{fwVer})$ *Floor Consistency: The floor never exceeds the running version.*
O	Safety	$\models \mathbf{G}\,(\text{now}\,(\text{phase}{=}\text{Applied}) \Rightarrow \text{now}\,\text{reset})$ *Reset Ordering: Mandates reset prior to capsule application.*
L	Liveness	$\models \mathbf{G}\,(\text{now}\,\text{good} \Rightarrow \mathbf{F}\,\text{now}\,(\text{phase}{=}\text{Applied}))$ *Responsiveness: Valid new capsules are guaranteed to apply.*

Latent Ambiguity: The Apply Guard (G)

Theorem (Apply Guard G)

$$\vDash \mathbf{G}\Big(\big(\mathsf{now}\,(\mathsf{phase}{\neq}\mathsf{Applied}) \wedge \mathbf{X}\ \mathsf{now}\,(\mathsf{phase}{=}\mathsf{Applied})\big) \ \Rightarrow\ \mathsf{now}\,(\mathsf{sigOK} \wedge \mathsf{lsv} \leq \mathsf{capVer})\Big)$$

What Informal Prose Claims

"When firmware is in the Applied state, the installed version is strictly newer than before."

Lean Kernel Proof Rejection:

- This model assigns fwVer := capVer and lsv := capVer.
- In the **post-state**, lsv = capVer.
- The acceptance guard belongs to the **pre-state**.

The Precision of Formal Proof

- EDK II checks the effective LSV floor, not fwVer directly.
- Equality with the floor is accepted.
- The model specializes the update policy to lsv := capVer.

Responsiveness: Progress Without Fairness

Theorem (Liveness / Responsiveness L)

$$\vDash \ \mathbf{G}\,(\text{now}\,\text{good} \ \Rightarrow \ \mathbf{F}\,\text{now}\,(\text{phase}{=}\text{Applied}))$$

$$\text{good} \triangleq (\text{phase}{=}\text{Staged} \wedge \text{certOK}(\text{digest}) \wedge \text{lsv} \leq \text{capVer})$$

Why Standard Liveness Needs Fairness:

- Usually requires weak/strong fairness to prevent a non-deterministic scheduler from starving actions.

Our Result: Zero Fairness Axioms!

- Good-path firmware execution is **deterministic up to the sink**.
- Once staged, intermediate steps are the *sole enabled transition*.

Deterministic 3-Step Progress Witness

$$\text{Staged} \xrightarrow{\text{Step 1}} \text{PostReset} \xrightarrow{\text{Step 2}} \text{Auth} \xrightarrow{\text{Step 3}} \text{Applied}$$

- Field preservation keeps digest, lsv, and capVer stable.
- At Auth, REJECT guard is false; APPLY is uniquely forced.

Trust Base: Kernel Verification with Zero ‘sorry‘

The Assurance Guarantee:

- Tactics & elaborators are untrusted.
- Lean 4 kernel independently verifies proof terms.
- No escape hatches (`sorry`, `admit`, `unsafe`).

Audited Axiom Dependency:

- `propext` (Propositional extensionality)
- `Quot.sound` (Quotient soundness)
- *No* Axiom of Choice (`Classical.choice`) needed!

Kernel Output (`#print axioms`)

```
'safety_authentic'
  depends on: [propext]
'antirollback_step'
  depends on: [propext, Quot.sound]
'antirollback_global'
  depends on: [propext, Quot.sound]
'apply_guarded'
  depends on: [propext]
'reset_first'
  depends on: [propext]
'responsiveness'
  depends on: [propext, Quot.sound]
```

Safe Rust Prototype: Conditional Refinement

Decision-Logic Prototype:

- Safe Rust function `advance(&mut State)`.
- Translated via **Charon / Aeneas** into monadic Lean 4.

Refinement Lemma

$$\forall s.\ s \twoheadrightarrow \mathrm{advance}(s)$$

Every abstract `advance` step is a legal model transition.

Guarantees Transfer as Free Corollaries

- **Authenticity on Code:** `safety_authentic_det`
- **Responsiveness on Code:** `responsiveness_det`

Concrete bridge: `CapsuleBridge.advance_refines` relates extracted Rust to abstract `advance`, assuming `VerifierRefines`.

Scope: Prototype decision logic only—not EDK II, PKCS#7, parsing, flash I/O, or hardware.

Complementary Validation: TLA+, TLC, and Boundary Probing

TLC Model Checking

- Transcribed model to TLA+ (157 states).
- Exhaustive state exploration of safety properties in baseline configuration.

Power-Fail Boundary Probe (`MC_crash.cfg`)

- Non-deterministic crash/power loss action.
- **Finding:** Safety (S, R_1, G, O) survives crashes.
- Responsiveness (L) fails under infinite crash loops (starvation).

TLAPS + Z3 Deductive Proofs

- Re-proved inductive invariants $(\mathrm{TypeOK} \wedge \mathrm{InvSig} \wedge \mathrm{InvReset})$.
- Discharged over unbounded integers $(\mathbb{N})$ via Z3.
- Provides a completely disjoint toolchain cross-check.

Summary & Takeaways

- **Core Achievements:**
  - First machine-checked LTL formalization of the UEFI Capsule Update process.
  - Proved seven named guarantees: S, R_1, R_2, F, G, O, and L.
  - Exposed temporal placement requirement for anti-rollback guards.
  - Related a Safe Rust decision-logic prototype through Aeneas and an explicit verifier assumption.
- **Key Takeaway:**
  - Lightweight interactive theorem proving (300 lines of Lean) pays massive dividends on security-critical firmware state machines.

Open Source Artifacts

GitHub Repository: `https://github.com/vincentjzimmer/formal` (`capsule-ltl/` directory)

Thank You! — Questions?